

Effets de bord réversibles et monades, dans le système de présentation Slipshow

Paul-Elliot Anglès d'Auriac

Slipshow est un logiciel de présentation, qui se focalise sur un style de discours plus « continu » que les diapositives. Très versatile, il permet à l'auteur de la présentation de déclencher de nombreuses actions à chaque étape, voire même d'en définir lui-même. Cela pose un défi d'ingénierie : comment « inverser » l'exécution d'une étape, quand le présentateur souhaite revenir en arrière, tout en gardant un code simple, maintenable et extensible ? Dans cet article, nous verrons la solution utilisée par Slipshow, basée sur le patron de conception des monades, et son utilisation rendue aisée par le langage de programmation OCaml.

1 Slipshow et le retour en arrière

Slipshow [Ad21] est un logiciel de présentation, à l'instar de Beamer, LibreOffice Impress ou Reveal.js. Il diffère cependant fondamentalement de ces exemples, en ce qu'il s'éloigne du concept de diapositives (ou « slides » en anglais).

L'inspiration principale de Slipshow est la présentation au tableau. Lors d'un cours au tableau, il n'est pas rare de séparer le tableau en deux, et de n'effacer qu'une moitié à la fois. Cela assure qu'au moins une moitié du contenu passé sera accessible pour le public, en cas de distraction momentanée.

Au contraire, si une déconcentration intempestive survient peu avant le passage à la diapositive suivante, le spectateur distrait manquera irrévérablement la fin de la diapositive, l'empêchant potentiellement de comprendre la suite de l'exposé.

Le concept de base d'une présentation Slipshow n'est pas le slide, mais le « slip ». Conceptuellement, un slide est un carré de contenu, du même format que l'écran le projetant. Un slip, lui, a la même largeur que l'écran, mais une longueur verticale arbitraire. Afin de montrer le contenu d'un slip, l'écran peut « glisser » pour en révéler les parties non visibles, à la manière d'un navigateur permettant le déroulement d'une page web.

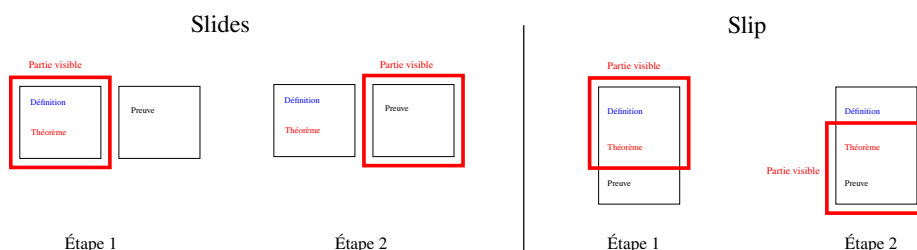


Figure 1. Le carré rouge représente ce qui est visible à l'écran. À gauche, la transition d'un slide à un autre est immédiate. À droite, le déroulement d'un slip est continu.

1.1 Contenu statique et dynamique

Bien que les slips forment la genèse du projet Slipshow, celui-ci s'est considérablement agrandi et supporte des présentations par diapositives, du style de Prezi, voire même utilisant des animations telles que celles souvent utilisées dans les vidéos de vulgarisation scientifique. Une présentation Slipshow consiste en deux parties entrelacées dans le fichier source : une description du contenu statique ; et une description de la dynamique de la présentation :

```
{.theorem}
Ceci est un théorème

{pause}

- premier point {pause}
- deuxième point
```

Le *contenu statique* est sous forme textuel, utilisant une syntaxe spéciale pour étiqueter des blocs d'une sémantique particulière (énumération à points, théorème, ...), le système s'occupant de la mise en page à partir de ces étiquettes.

Le *contenu dynamique* d'une présentation est ici défini dans le corps du texte, à nouveau à l'aide d'une syntaxe spéciale. Nous avons deux actions **pause**, qui à leur activation révèlent le contenu qui suit.

Contrairement à la plupart des systèmes de présentation de diapositives, l'ensemble des actions possibles pour Slipshow est très riche. En voici un petit aperçu :

- **scroll-to=ID** déroule l'écran jusqu'à ce que l'élément désigné soit entièrement visible.
- **focus=ID** zoome sur l'élément désigné.
- **static=ID** insère un élément caché à sa position originelle.
- **play-media=ID** joue le fichier audio ou vidéo désigné.
- **draw=ID** dessine une annotation préalablement enregistrée.
- **speaker-notes** affiche des notes de présentateur dans la vue présentateur¹.
- **exec** exécute un script défini par l'utilisateur.

Une volonté assumée de Slipshow est de conserver cette richesse, au prix d'un développement plus contraint. Cette dernière action, **exec**, offre de nombreuses possibilités à l'orateur, mais enlève du contrôle à Slipshow sur ce qui peut se passer lors d'une présentation. Un exemple typique est la gestion du retour en arrière.

1.2 Le retour en arrière

La complexité du système d'actions de Slipshow pose un défi en termes de programmation : comment gérer le retour en arrière ? Lors d'une présentation, il est fréquent que l'orateur souhaite revenir à une étape précédente, par exemple pour clarifier un point ou répondre à une question.

Dans le cas de présentations classiques, une manière classique de résoudre le problème est la suivante. Nous représentons l'« état » d'une étape d'une présentation : par exemple le numéro de diapositive courante et l'ensemble des éléments affichés dans la diapositive. Nous définissons aussi une fonction **render**, qui synchronise la vue de l'utilisateur et la valeur de l'état, ainsi qu'une fonction **next** qui passe à l'état suivant. La fonction **render** est appelée à chaque changement d'état courant.

Pour avancer dans une présentation tout en permettant le retour en arrière, il suffira alors de stocker dans une pile les anciens états. Si la présentation a débuté par l'état S_0 , et que l'on passe à l'état suivant $S_1 = \text{next}(S_0)$, on empile l'état S_0 dans la pile. Revenir en arrière consiste simplement à remplacer l'état courant par l'état en tête de pile (en le dépilant).

1. Une fenêtre séparée, visible seulement par l'orateur, contenant un chronomètre et des notes.

Cependant, cette approche pose plusieurs problèmes pour Slipshow. Le premier problème est que la notion d'état est limitante à un type prédéfini, et non extensible. La quantité de données à inclure dans l'état d'une étape d'une présentation Slipshow n'est pas fermée : outre les nombreux états des actions autorisées, et l'état du document HTML, l'orateur peut être amené à définir son propre état via l'exécution de script.

Le second problème est que, dans le modèle décrit ci-dessus, la fonction **render** ne permet pas de contrôler les transitions. Or, dans le cas d'une présentation, les transitions sont importantes, et il est naturel de pouvoir programmer que le passage de l'état S_0 à l'état S_1 soit différent du passage de S_2 à S_1 , correspondant à un retour en arrière.

1.3 Des états aux transitions

Pour les raisons énoncées ci-dessus, Slipshow n'empile pas les états précédents pour pouvoir revenir en arrière. Il donne la part belle, non aux états, mais aux transitions entre états.

On pourrait donc imaginer avoir deux fonctions, **next** mais aussi **previous** : l'une pour avancer d'une étape, l'autre pour reculer. Cependant, la multiplicité des actions possibles, ainsi que leurs interactions, rend la définition de **previous** ardue. Par exemple, l'action **scroll-to=ID** n'a pas d'action inverse évidente : il faut se souvenir de la position de départ pour pouvoir y revenir. L'exécution d'un script utilisateur peut modifier l'état de la présentation de manière arbitraire, rendant à priori impossible la définition d'une action inverse générique. Finalement, inverser de multiples actions exécutées successivement doit se faire dans un ordre précis, au risque de ne pas retrouver l'état initial.

Cette complexité est principalement en termes de lisibilité et maintenabilité du code, et non algorithmique. Il est facile de se rendre compte que théoriquement, l'on peut définir une fonction pour aller en avant dans la présentation, et une fonction pour retourner en arrière. Cependant, une manière naïve de procéder risquerait d'être difficile à maintenir et à appréhender, avec une logique dupliquée et des risques d'incohérences.

Par exemple, lors de l'intégration au code de Slipshow d'une action supplémentaire, il faudrait penser à implémenter son action inverse dans la fonction de retour en arrière. De plus, si une action est modifiée, il faudrait penser à modifier son action inverse en conséquence. Un système d'analyse statique serait difficile à mettre en place, on ne pourrait donc compter que sur la vigilance du développeur.

1.4 Calculer sa propre annulation

La version 0.1.0 de Slipshow a consisté en une réécriture complète du logiciel, dans le langage OCaml. Cette réécriture a été l'occasion de repenser le système de gestion du retour en arrière. Plusieurs versions et de nombreuses fonctionnalités plus tard, cette approche a prouvé son efficacité.

Au lieu de définir une fonction **next** ainsi qu'une fonction **previous**, on définit uniquement **next**, mais celle-ci, en plus de modifier l'état courant, retourne la fonction pour annuler les modifications. Ces fonctions d'annulation seront mises dans une pile au fur et à mesure de l'avancée de la présentation.

Ainsi, à chaque étape, si l'on veut progresser on définira **previous := next()** que l'on empilera dans la pile. Si l'on veut revenir à l'étape précédente, il suffira de dépiler et exécuter la fonction en tête de pile.

Il ne reste donc plus qu'à définir la fonction **next**, ce qui est un problème de taille ! Mais nous allons nous armer d'un patron de conception qui s'avèrera très utile, et permettra de définir la fonction **previous** à retourner par combinaison d'annulations « atomiques ».

Dans cet article, nous commençons par introduire le patron de conception des « monades ». Les différents concepts aboutissant à celui de monades sont exemplifiés en utilisant le langage de programmation OCaml.

L'introduction aux monades rend l'article autonome, mais sa présentation reste brève afin de se concentrer sur la deuxième partie de ce papier : l'application de ce patron de

conception au problème initial du « retour en arrière », à l'aide d'une monade originale : la monade *Annulation*, qui constitue la contribution principale de ce papier.

2 Monades et programmation

Une monade, en programmation, est un patron de conception [Wad93], une manière d'organiser son code pour obtenir certains bénéfices. Nous allons présenter ce patron de conception en passant par deux étapes intermédiaires : les *foncteurs* et les *foncteurs applicatifs*. Cette terminologie, et les définitions qui suivront sont inspirées de la théorie des catégories, bien qu'il ne soit pas nécessaire de connaître cette théorie pour profiter des bienfaits de ce patron de conception.

Les monades permettent de faire des calculs avec, non des valeurs, mais des *valeurs spéciales*, tout en faisant « comme si » c'étaient des valeurs normales. La notion de « spéciale » dépendra du problème que l'on souhaite résoudre, mais nous prendrons dans cette section l'exemple de valeurs dont le calcul a pu produire une, ou plusieurs, erreurs.

```
type 'a t = Ok of 'a | Error of string list
```

Nous allons voir comment « passer » de calculs sur des valeurs (c'est à dire, des fonctions de type `'a -> 'b`) en des calculs sur des valeurs spéciales (c'est à dire, des fonctions de type `'a t -> 'b t`).

2.1 Les foncteurs

Définition 1 (Foncteur). Un *foncteur* est une structure :

```
type 'a t
val map : ('a -> 'b) -> ('a t -> 'b t)
```

La fonction `map` doit satisfaire deux contraintes de compatibilité :

- `map (fun (x : 'a) -> x)` et la fonction identité de type `'a t -> 'a t` coïncident.
- Si `f : 'a -> 'b` et `g : 'b -> 'c`, et `(o)` dénote la composition de fonction, alors les fonctions `map (g o f)` et `(map g) o (map f)` coïncident.

La fonction `map` passe d'un calcul de valeurs normales, à un calcul de valeurs spéciales. Dans notre exemple de « valeurs qui peuvent être des erreurs », nous pouvons définir une telle fonction comme suit :

```
let map f = function
| Ok x -> Ok (f x)
| Error _ as x -> x
```

En voici un exemple d'utilisation de `map`, dans le cas où l'on utilise une fonction `read_file : string -> string t` (où `'a t` représente nos « valeurs avec erreurs » ici) :

```
let read_and_sanitize path =
  let content = read_file path in
  let mouline c = c |> sanitize |> convert in
  map mouline content
```

Nous avons défini notre calcul, `mouline`, dans l'espace des valeurs normales, mais nous l'appliquons à une valeur pouvant être une erreur à l'aide de `map`. Cependant, cela complique le code, par rapport à une « inlinée » de `mouline`. Heureusement, la syntaxe OCaml permet de faire mieux.

2.1.1 Support syntaxique en OCaml

Si on définit fonction nommée `(let+)`, en OCaml va traiter `let+ <var> = <value> in <body>` comme étant du sucre syntaxique pour `(let+) <value> (fun <var> -> <body>)`. Ce sucre syntaxique permet d'améliorer l'exemple précédent :

```
let (let+) x f = map f x

let read_and_sanitize path =
  let+ content = read_file path in
  content |> sanitize |> convert
```

On peut voir que la définition de `read_and_sanitize` correspond à celle que l'on aurait écrit si `read_file` avait eu le type `string -> string`, à un unique caractère près : le `+`!

2.1.2 Limitations

Cependant, si les foncteurs permettent de « transposer » un calcul (représenté par une fonction à une variable) au contexte `'a t`, ils sont rapidement limités : leur définition est trop générale pour transposer des fonctions à multiples arguments, sous formes curryfiées. En effet, étant donnée une fonction `'a -> 'b -> c`, on peut grâce à `map` obtenir une fonction de la forme `'a t -> ('b -> 'c) t`, et non ce que l'on souhaiterait : `'a t -> 'b t -> 'c t`.

Concrètement, dans l'exemple précédent, nous ne pouvons pas combiner deux résultats de la fonction `read_file`.

2.2 Les applicatifs

Définition 2 (Applicatif [MP08]). Un *foncteur applicatif* est un foncteur avec les fonctions additionnelles suivantes :

```
val pure : 'a -> 'a t
val pair : 'a t -> 'b t -> ('a * 'b) t
```

De même que `map` dans la définition des foncteurs, ces deux fonctions doivent satisfaire des propriétés supplémentaires pour correspondre à ce qu'on attend d'elles :

- `map (fun (x, y) -> f x, g y) (pair u v)` et `pair (map f u) (map g v)` sont équivalents à observation près.
- `map fst (pair u (pure ()))` et `u` sont équivalents à observation près.
- `map snd (pair (pure ()) v)` et `v` sont équivalents à observation près.
- `map recombine (pair u (pair v w))` et `pair (pair u v) w` sont équivalents à observation près, où `recombine` est `fun (a, (b, c)) -> ((a, b), c)`.

Dans la sous-section 2.1.2, nous avons vu que nous ne pouvions transposer un calcul sous la forme d'une fonction à deux variables : nous ne pouvions transformer `'a -> 'b -> 'c` en `'a t -> 'b t -> 'c t`.² Grâce à `pair`, c'est maintenant possible :

```
let map2 f x1 x2 = map (fun (x1, x2) -> f x1 x2) (pair x1 x2)
```

Reprenant notre exemple des valeurs à erreurs, nous pouvons l'étendre en un foncteur applicatif :

```
let pure x = Ok x
let pair x y = match x, y with
| Ok vx, Ok vy -> Ok (vx, vy)
| Error ex, Error ey -> Error (ex @ ey)
| Error e, Ok _ | Ok _, Error e -> Error e
```

2. La limitation à appliquer `map` deux fois était qu'il fallait une version de `map` de type `('a -> 'b) t -> 'a t -> 'b t`. C'est en réalité une définition équivalente à celle donnée dans cet article.

2.2.1 Support syntaxique en OCaml

De même que OCaml améliore l'utilisation de `map` via `(let+)`, il supporte un sucre syntaxique similaire via `(and+)` : `let+ <var1> = <e1> and+ <var2> = <e2> in <body>` est équivalent à `(let+) (fun (<var1>,<var2>) -> <body>) ((and+) <e1> <e2>)`. Nous pouvons donc écrire :

```
let (and+) x y = pair x y

let read_and_combine path1 path2 =
  let+ content1 = read_file_and_sanitize path1
  and+ content2 = read_file_and_sanitize path2 in
  String.concat content1 content2
```

On peut voir que la définition de `read_and_combine` correspond à celle que l'on aurait écrit si `read_and_sanitize` avait eu le type `string -> string`, à deux uniques caractères près : les deux `+` accolés aux `let` et `and` !

2.2.2 Limitations

Les applicatifs comme patron de conception sont déjà utiles. Ils permettent de calculer avec des valeurs « spéciales », tout en maintenant une connaissance statique de toutes les valeurs spéciales à partir desquelles nous calculons. Ce patron de conception permet par exemple la génération de « man pages » automatique pour une interface en ligne de commande définie déclarativement [Bü].

Cependant, il manque encore un ingrédient crucial pour pouvoir calculer avec des valeurs spéciales, de manière générale : la possibilité de « transposer » un calcul produisant une valeur spéciale. Par exemple, si nous souhaitons transposer `read_and_sanitize` pour l'utiliser sur une valeur pouvant être une erreur. En d'autres termes, nous souhaiterions transposer un calcul de type `'a -> 'b t` en un calcul de type `'a t -> 'b t`.

2.3 Des applicatifs au monades

Définition 3 (Monades). Une *monade* est un foncteur applicatif muni de la fonction additionnelle suivante :

```
val join : 'a t t -> 'a t
```

Une monade doit également satisfaire des règles additionnelles : l'associativité de `join`, et sa compatibilité vis à vis de `pure` : `a = join (pure a) = join (map pure)`.

Il n'est pas encore complètement clair comment `join` améliore nos possibilités de transposition. C'est via la fonction `bind` : `('a -> 'b t) -> 'a t -> 'b t` définie via `join` par `let bind f x = join (map f x)` qui permet cette transposition. Notons que `bind` permettrait de définir `map` et `join`, et c'est d'ailleurs une définition alternative aux monades !

Dans notre exemple des valeurs potentiellement erreurs :

```
let join f = function
| Ok (Ok x) -> Ok x
| Ok (Error e) | Error e -> Error e
```

Ainsi, nous définissons un calcul pouvant échouer, et pouvons l'appliquer à une valeur pouvant être elle-même une erreur, comme si c'était une valeur normale.

En réutilisant les `let`-bindings d'OCaml, nous pouvons définir un calcul complet « comme s'il était dans des valeurs normales », mais l'appliquer à des valeurs pouvant retourner des erreurs !

```

let (let*) x f = bind f x

let compute path =
  let* content1 = read_file path in
  let* new_path = extract_path content1 in
  let* content2 = read_file new_path in
  compute content2

```

Ceci conclut notre brève introduction des monades. Il existe de nombreuses monades « classiques », très utilisées telle la monade `Result` pour les valeurs dont le calcul a pu échouer avec une erreur [Got], la monade `Option` pour les valeurs dont le calcul s’est arrêté [Got], mais aussi des monades pour la programmation fonctionnelle réactive [Bou, JSG], la monade `IO` pour encoder les effets de bord [HPJ], la monade `State` pour propager un état dans un calcul [Got], la monade `List` pour un calcul non-déterministe [Got], une monade utile pour le parsing [Eli]...

Cependant, la monade que nous allons présenter dans la section suivante est une monade moins connue, et pourtant particulièrement utile dans son cas d’usage.

3 La monade *Annulation* pour le retour en arrière

Revenons à notre problème initial. Nous avons vu dans la section `TODO` la façon dont `slipshow` passe d’une étape à la suivante : en appelant une fonction `next`, qui en sus de ses effets de bords nécessaires à l’avancée de la présentation, retourne une fonction permettant d’annuler ces effets de bords.

Nous en étions donc à voir comment définir `next` sans introduire un excès de complexité, malgré les nombreuses actions offertes par `Slipshow`. Pour cela, nous allons utiliser le patron de conception des monades, que nous venons de définir.

3.1 La monade *Annulation*

La fonction `next` est définie à l’aide d’une monade dont les « valeurs spéciales » sont des valeurs dont le calcul a produit des effets de bord, mais qui possèdent une fonction pour annuler ces effets de bord :

```

type 'a io = unit -> 'a
type 'a undoable = { value : 'a ; undo : unit io }
type 'a t = 'a undoable io

```

Par exemple, voilà comment changer une référence mutable, tout en donnant la possibilité d’annuler ce changement :

```

let set (x : 'a ref) (v : 'a) : unit t = fun () ->
  let undo =
    let old_x = !x in
    fun () -> x := old_x
  in
  let value = x := v in
  { value ; undo }

```

Afin de définir une monade, il nous faut une fonction `return` pour plonger nos valeurs normales, et une fonction `bind` pour séquencer des valeurs de valeurs annulables ; satisfaisant les règles monadiques.

```

let return x = fun () -> { value = x; undo = fun () -> () }

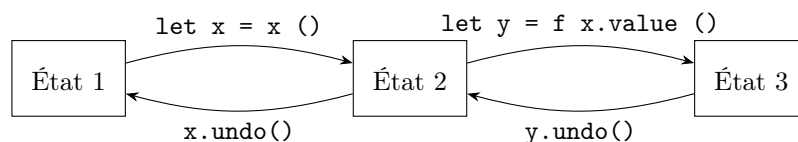
```

```

let bind f x = fun () ->
  let x = x () in
  let y = f x.value () in
  let undo () = y.undo () ; x.undo () in
  let value = y.value in
  { value ; undo }

```

L'ordre d'appel de `y.undo` et `x.undo` dans la fonction `bind` est le point crucial de cette monade. En effet, les effets de bord du calcul de `y` se passant après ceux de `x`, il faut les annuler dans l'ordre inverse, comme l'illustre la figure suivante :



On peut maintenant exprimer naturellement un calcul avec effets de bord, l'effectuer, puis inverser ces effets de bord.

```

let x = ref 0
let compute =
  let* () = set x 2 in
  set x 4

let res = compute () (* x passe de 0 à 2 puis à 4 *)
let () = res.undo () (* x passe de 4 à 2 puis à 0 *)

```

3.2 L'annulation dans le code source de Slipshow

La monade *Annulation* est omniprésente dans le code source de Slipshow. Un certain nombre de « primitives » annulables sont définies : pour changer la classe d'un élément, définir son style, déplacer la vision courante de la présentation...

Slipshow combine ensuite séquentiellement ces valeurs, en utilisant `let*` pour garder une lisibilité similaire à celle qu'on aurait en l'absence de monade.

Cette organisation a permis de corriger de nombreux bugs, ainsi que d'ajouter de nombreuses fonctionnalités, sans efforts particuliers.

3.3 L'annulation dans les scripts utilisateur

Une des difficultés récurrentes provient de la possibilité pour un auteur de présentation de définir ses propres scripts, à exécuter à une étape de la présentation, via l'action `exec`. Comment la fonction d'annulation de cette action peut-elle être définie ?

Slipshow se retourne vers l'impératif pour cela : il définit une variable mutable, initialisée à la liste vide. Via l'espace de nom `slip`, il donne ensuite à l'auteur du script une API JavaScript pour modifier le DOM (par exemple, changer le style d'un élément). Lors de l'appel à une fonction de l'API, la variable mutable est augmentée d'une fonction d'annulation correspondante.

```

let undos = ref []

let set_class_js el class b = (* Appellable en JavaScript, *)
  let compute = set_class el class b in (* via slip.set_class *)
  undos := (compute()).undo :: !undos

```


L’auteur du script définit donc, s’il utilise convenablement l’API, une fonction d’annulation de manière transparente.

```
my_elements.forEach(el => {  
  slip.set_class(el, "highlight", true);  
});
```

Dans le script utilisateur ci-dessus (défini dans la source d’une présentation et exécuté avec l’action `exec`), les différents appels à `slip.set_class` ajoutent chacun une entrée à la liste de fonctions `undos`.

3.4 La monade *Annulation* et les animations

Une subtilité de la monade *Annulation* est son interaction avec la monade *Fut* représentant la programmation concurrente. En effet, en particulier dans le contexte d’une présentation, l’orateur voudra définir des animations pour illustrer ses concepts. On peut donc modifier le type de la monade en :

```
type 'a undoable = { value : 'a ; undo : unit Fut.t io }  
type 'a t = 'a undoable Fut.t io
```

Notons que le type du champ `undo` est aussi modifié : annuler une animation pourrait être une animation aussi ! De là à définir un transformateur de monade [LHJ95], il n’y a qu’un pas, que nous ne franchirons pas dans cet article.

3.5 La monade *Annulation* et les erreurs

Pour notre dernier exemple d’utilisation de la monade *Annulation*, considérons son utilisation conjointe avec la monade *Result*.

```
type 'a undoable = { value : 'a ; undo : unit result io }  
type 'a t = 'a undoable result io
```

Illustrons son intérêt dans un contexte différent de celui de Slipshow. Lors d’une modification d’une base de données, il n’est pas rare de mettre la base à jour en utilisant de multiples étapes. Si une de ces étapes échoue, il y a un risque d’obtenir une base dans un état incohérent. La monade *Annulation* permet de s’assurer que l’on aura accès à une fonction permettant (si elle n’échoue pas) de rétablir la base dans son état cohérent initial. Cette utilisation s’apparente à la notion de saga définie dans [GMS87].

4 Conclusion

Nous sommes partis d’un cas concret, la programmation d’un support de présentation généraliste. Cela nous a mené à des concepts abstraits de théorie des catégories, appliqués comme patrons de conception. Ces patrons de conception ont finalement pu être appliqués à notre cas d’étude, nous permettant de programmer presque normalement, tout en obtenant gratuitement une fonction pour annuler nos effets de bord.

De plus, ce patron de conception a tenu bon face aux différentes contraintes et extensions que nous avons : les scripts utilisateurs en JavaScript, les animations. Il reste d’autres défis à venir, comme celui d’arrêter, annuler une animation en cours, puis la reprendre avant qu’elle ait entièrement fini... cas particuliers pouvant bien arriver dans le cadre de Slipshow !

Références

- [Ad] Paul-Elliot ANGLÈS D'AURIAC : Support de présentation de : Infinite computations in algorithmic randomness and reverse mathematics. <https://choum.net/panglesd/slides/slides-js/slides.html>.
- [Ad21] Paul-Elliot ANGLÈS D'AURIAC : Le projet Slipshow. <https://github.com/panglesd/slipshow>, 2021.
- [Bou] Frédéric BOUR : Lwd : a "lightweight document" library. <https://github.com/let-def/lwd>.
- [Bü] Daniel BÜNZLI : Cmdliner : Declarative command line argument parsing for ocaml. <https://erratique.ch/software/cmdliner>.
- [Eli] Spiros ELIOPOULOS : Angstrom : Parser combinators built for speed and memory-efficiency. <https://github.com/inhabitedtype/angstrom>.
- [GMS87] Hector GARCIA-MOLINA et Kenneth SALEM : Sagas. *ACM SIGMOD Record*, 16(3):249–259, décembre 1987.
- [Got] Ivan GOTOVCHITS : Monads : Monads transformers for most common monads. <https://ocaml.org/p/monads/2.5.0/doc/monads/Monads/Std/index.html>.
- [HPJ] Paul HUDAK, John PETERSON et Fasel JOSEPH : A gentle introduction to haskell, version 98. <https://www.haskell.org/tutorial/io.html>.
- [JSG] LLC JANE STREET GROUP : Incremental : A library for incremental computations. <https://github.com/janestreet/incremental>.
- [LHJ95] Sheng LIANG, Paul HUDAK et Mark JONES : Monad transformers and modular interpreters. In *Proceedings of the 22nd ACM SIGPLAN-SIGACT symposium on Principles of programming languages - POPL '95*, POPL '95, page 333–343. ACM Press, 1995.
- [MP08] Conir MCBRIDE et Ross PATERSON : Applicative programming with effects. *Journal of Functional Programming*, 18(1):1–13, janvier 2008.
- [Wad93] Philip WADLER : *Monads for functional programming*, page 233–264. Springer Berlin Heidelberg, 1993.